FT-991a Companion: User Manual

Oct 30, 2026 · @Charles

Overview

FT-991a Companion is a set of programs for operating a Yaesu FT-991A from another computer. The **panel** (`ft991a-panel.py` , or the installer) controls the radio and carries its audio over the network. The **FreeDV Audio Bridge** (`audio-bridge.py`) carries a radio's audio between two computers, so FreeDV or WSJT-X can run on one machine while the radio sits on another. The **CW window** sends text and keys the radio from a HaliKey MIDI key or paddles, through a small keying daemon on the radio's computer. It is free software under the GNU General Public License, version 3 or later.

The system has two computers and two kinds of traffic:

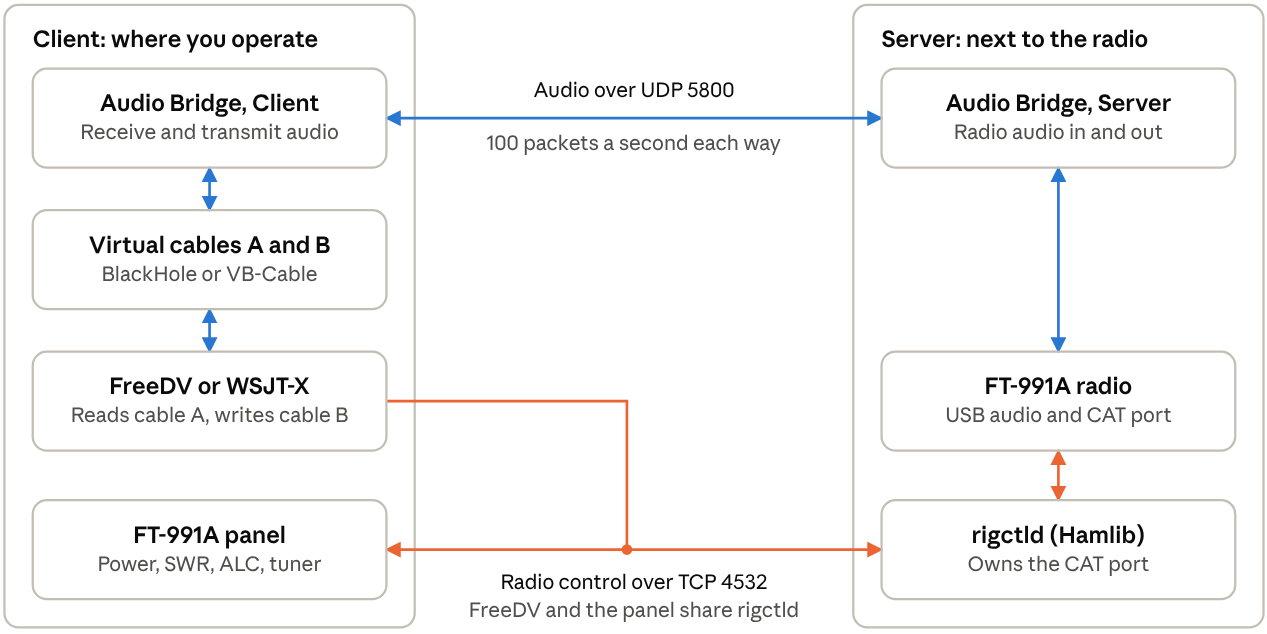
- **Server:** the computer next to the radio. It owns the radio's USB audio device and its serial (CAT) port.
- **Client:** the computer where you operate. It runs FreeDV or WSJT-X, the bridge in Client mode, and the panel.
- **Audio:** the bridge sends 100 small UDP packets a second each way, on port 5800.
- **Radio control:** Hamlib's `rigctld` on the server answers CAT commands over TCP port 4532. FreeDV, the panel and your logger all connect to it.

The bridge moves audio only. It does not create audio devices (the client needs virtual audio cables, see Audio devices and virtual cables) and it does not key the transmitter.

Part	Runs on	What it does
Audio Bridge, Server role	Server	Sends the radio's receive audio to the client; plays the client's transmit audio into the radio.
Audio Bridge, Client role	Client	Plays receive audio into a virtual cable that FreeDV reads; captures FreeDV's transmit audio from a second virtual cable.
rigctld (Hamlib)	Server	Gives several programs shared, orderly access to the radio's CAT port.

Part	Runs on	What it does
FT-991A panel	Client	Shows frequency, power, SWR and ALC; sets mode, power and tuner.
Diagnostic tools	Either	Measure packet loss, record audio, and test rigctld.

Audio and radio control cross the network on separate paths



architecture · client and server, two network paths

Audio crosses between the two bridges on UDP port 5800. FreeDV and the panel both reach the radio's CAT port through rigctld on TCP port 4532.

This manual reflects bridge version 2026-10-26a.

What you need

You need a radio with a USB sound interface, two computers that can reach each other, and Python 3 on both. The FT-991A has a built-in USB audio device, which appears on the server as a "USB Audio CODEC".

Python packages (both computers)

- `sounddevice` for audio. It needs the PortAudio library.

- `opuslib` plus the native Opus library, only if you want the Opus codec. PCM works without them.
- `tkinter` for the windows (the panel and the bridge GUI). It is included with Python on Windows and macOS.
- `python-rtmidi`, only if you use a HaliKey MIDI key or paddles in the CW window. The panel runs it in a helper process, so a crash in it cannot take the panel down.
- `pyserial`, only on the radio's computer, for `keying-daemon.py`. The panel itself does not use it.

Platform	Install
Linux	`pip install sounddevice opuslib numpy` and `sudo apt install libportaudio2 libopus0 python3-tk`
macOS	`pip install sounddevice opuslib numpy` and `brew install opus`. If Python cannot find the Opus library on Apple Silicon, set `DYLD_FALLBACK_LIBRARY_PATH=/opt/homebrew/lib`.
Windows	`pip install sounddevice opuslib numpy`, and put a 64-bit `opus.dll` in the same folder as the scripts. It must match your Python (64-bit). The scripts add their own folder to the DLL search path.

Virtual audio cables (client only)

The client needs two virtual cables, so receive audio and transmit audio travel on separate paths.

Platform	Cables
macOS	`brew install blackhole-2ch blackhole-16ch`, then restart or run `sudo killall coreaudiod`.
Windows	VB-Cable plus a second cable (the VB-Cable A+B pack).

Platform	Cables
Linux	Two PulseAudio or PipeWire null sinks, created with `pactl load-module module-null-sink`.

Radio control and network

- Hamlib, which provides `rigctld` (server) and, optionally, `rigctl`. See Radio control with Hamlib.
- A network path between the computers. This installation uses Tailscale, so no port forwarding is needed.

Use the same version of the bridge on both computers. Each prints `Bridge version ...` in its log when it starts.

Quick start

Set up the server first, then the client, then FreeDV. Use PCM for the first test; it is the simplest and the least likely to disturb the FreeDV signal.

1. **Server: power on the radio.** Connect its USB cable and turn it on, so its audio device exists before the bridge starts.
2. **Server: start the bridge.** Run `python3 audio-bridge-gui.py`. Choose Mode **Server**. Set Capture to the radio's input and Playback to the radio's output. Set Transmit codec to **pcm**. Press **Start**. The log shows `Listening on UDP 0.0.0.0:5800` and a version line.
3. **Client: install the virtual cables** (see Audio devices and virtual cables).
4. **Client: start the bridge.** Run the same script. Choose Mode **Client** and enter the server's address in Server host/IP. Set Capture to cable B and Playback to cable A. Set Transmit codec to **pcm**. Press **Start**.
5. **Check the link.** Both windows show **Connected** in green. The server's "RX audio from radio" meter moves with the radio's receive audio, and so does the client's "RX audio to FreeDV" meter.
6. **Wire FreeDV.** Set "Input From Radio" to cable A and "Output To Radio" to cable B (see the wiring table).
7. **Set the transmit level.** With a dummy load connected, transmit and lower the server's "TX audio to radio" gain slider until the ALC reads idle and power output matches your setting.

8. **Check the counters.** After a few minutes, Lost should be well under 0.5% of RX packets (see Reading the status display).

The order of starting does not matter after the first run. If one end is restarted, the other resynchronises on its own.

Files and installation

Everything is plain Python or shell scripts. Copy the files to a folder on each computer (for example with `scp`) and run them from there. There is nothing to install beyond the packages listed above.

File	Put it on	Purpose
`audio-bridge.py`	Both	The engine, and the command-line program.
`audio-bridge-gui.py`	Both	The bridge window. It needs `audio-bridge.py` in the same folder.
`opus.dll`	Windows only	The Opus library, 64-bit, in the same folder as the scripts.
`ft991a-panel.py`	Client	The panel (FT-991a Companion), the program you run. It talks to the radio over the raw CAT connection or rigctld, and has its own audio engine, the Function tab and the CW window.
`fake-rigctld.py`	Either	A simulated rigctld, for trying the panel without a radio. fake-ft991a.py does the same for the Raw CAT connection, and fake-cwdaemon.py is a simulated cwdaemon for the CW window.
`udpseq.py`	Both	Packet-loss tester for the network path.
`wavstat.py`	Server	Analyses the bridge's debug recordings.
`rigtest.sh`	Server	Probes rigctld and prints one report.
`nettest.sh`	Server	Tests whether Hamlib's client can complete its handshake.
`radio-power.sh`	Server	Switches the radio on or off over CAT. It needs the serial port to itself.

File	Put it on	Purpose
`freedv-guard.sh`	Server	Saves FreeDV's audio device settings and puts them back if FreeDV overwrites them. Can also start FreeDV only once the radio's audio devices exist.
keying-daemon.py	Server	Keys the radio's second USB serial port for the CW window: text (it answers cwdaemon's commands) and timestamped key events. It replaces cwdaemon; run only one of them. Needs pyserial.
keying-daemon.service	Server	A systemd user service that runs the keying daemon in the background.
audio-bridge.service	Server	A systemd user service that runs the audio bridge in the background.

Settings files

- `~/.audio-bridge.json` holds the bridge window's settings: mode, addresses, codec options, gains, window size and position, and the audio devices you intend to use for each mode.
- `~/.ft991a-panel.json` holds the panel's host, port, window size and position, any meter correction points, the S9 reference, and whether the calibration buttons are shown, the CW window's settings (speed, calls, macros, the daemon addresses, the HaliKey device and port, the mode, the volume and pitch, and whether More is open), and auto_connect, which is on unless you set it to false.

Renamed files. Before version 1.1.2 the settings files were called `~/.freedv_audio_bridge.json` and `~/.ft991a_panel.json`, and the programs were `freedv_audio_bridge.py`, `freedv_audio_bridge_gui.py` and `ft991a_panel.py`. The programs still read the old settings files when the new ones do not exist. The systemd service was `freedv-audio-bridge` and is now `audio-bridge`: if you installed the old one, the CHANGELOG entry for 1.1.2 lists the steps to switch.

Deleting a settings file returns that program to its defaults. The files are written when you press Start, change a device, or close the window.

Keeping the two ends in step

The bridge prints `Bridge version 2026-10-26a ...` in its log at start, on both computers. If the two versions differ, or one end lacks a feature described here, copy the newer files over. Both files (`audio-bridge.py` and the GUI) must be updated together.

When you paste the output of a script into a chat, paste it into the chat only. Pasting it back into a terminal runs each line as a command.

Audio devices and virtual cables

The bridge only moves audio between devices that already exist. On the server those are the radio's own USB audio input and output. On the client you create two virtual cables, and FreeDV and the bridge connect to opposite ends of them.

Client wiring

Direction	Bridge (Client mode)	FreeDV audio settings
Receive: radio to FreeDV	Playback = **cable A**	Input From Radio = cable A
Transmit: FreeDV to radio	Capture = **cable B**	Output To Radio = cable B

Use your real speakers and microphone for FreeDV's other two slots (Output To Speakers and Input From Microphone).

Platform	Cable A	Cable B
macOS	BlackHole 2ch	BlackHole 16ch
Windows	VB-Cable ("CABLE Input" and "CABLE Output")	The second cable (for example "CABLE-B")
Linux	One null sink, for example `fdv_rx`	A second null sink, for example `fdv_tx`

On Linux, FreeDV reads receive audio from `fdv_rx.monitor` and writes transmit audio to `fdv_tx` ; the bridge captures from `fdv_tx.monitor` .

macOS notes

- In Audio MIDI Setup, set both BlackHole devices to 48000 Hz, and check that the output volume is at maximum and not muted.

- Allow microphone access for Terminal (or whichever app launches the bridge) and for FreeDV, in System Settings, Privacy & Security. Without it, capture silently returns silence.
- If the bridge's Capture meter stays flat while FreeDV transmits, check that FreeDV's Output To Radio is the same BlackHole device the bridge captures from.

Server devices (this installation)

Setting	Device
Capture (radio receive audio)	`alsa_input.usb-Burr-Brown_from_TI_USB_Audio_CODEC-00.analog-stereo [PulseAudio]`
Playback (radio transmit audio)	`alsa_output.usb-Burr-Brown_from_TI_USB_Audio_CODEC-00.analog-stereo [PulseAudio]`

Choose the entries whose names end in `[PulseAudio]` on a PulseAudio or PipeWire system. The same hardware also appears under other audio interfaces, and they can behave differently.

Why playback is stereo

The bridge carries mono audio but plays it to the output as stereo, with the same audio in both channels (setting Playback channels = 2, the default). When a PulseAudio device is selected by name, PipeWire does not copy a mono stream to both channels. The audio then reached only the left channel, and the radio, which was apparently listening on the right, heard nothing and produced no power. The stereo default fixes this. Leave it at 2 unless a device offers only one channel.

Capture reads one channel

The bridge captures the first (left) channel of a device. A program that writes only the right channel will appear silent. Set such programs to mono or both channels (this matters for WSJT-X).

When a device is missing

If a saved device is not present, for example because the radio is switched off, the dropdown shows `(not present) <device name>` and your saved choice is kept. Pressing Start then shows "Waiting for audio device..." and the bridge starts by itself when the

device appears. Press Stop to cancel. Only a device you choose yourself, or one the bridge has successfully started with, is saved.

Only one program should use the radio's audio device at a time. Close any local FreeDV that is using it before the server bridge runs.

The Audio Bridge window

Start it with `python3 audio-bridge-gui.py`. Settings are locked while the bridge runs, except the two gain sliders, which act immediately. The window keeps the size and position you leave it at.

Mode and network

Control	Meaning	Default
Mode	**Server** (at the radio) or **Client** (where FreeDV runs).	Client
Server host/IP	Client only: the server's address, for example its Tailscale address.	empty
UDP port	The port the server listens on and the client sends to. Must match on both.	5800
Bind address	Server only: the address to listen on. Use the Tailscale address to listen only on that network.	0.0.0.0 (all)

Audio devices

Control	Meaning
Capture	The input to read. Server: the radio's receive audio. Client: cable B, which carries FreeDV's transmit audio.
Playback	The output to write. Server: the radio's transmit input. Client: cable A, which carries receive audio to FreeDV.

Control	Meaning
Refresh	Rescans the audio devices. It also happens automatically every 3 seconds while a saved device is missing.

Codec and link

Control	Meaning	Default and range
Transmit codec	What this end sends: `pcm` (uncompressed) or `opus`. The other end decodes either automatically.	pcm recommended
Sample rate (Hz)	Must match on both ends. Opus accepts 8000, 12000, 16000, 24000, 48000.	48000
Frame (ms)	Audio per packet. Must match on both ends. Opus accepts 5, 10, 20, 40, 60. 20 ms halves packet overhead at the cost of 10 ms more delay.	10
Opus bitrate (kbps)	Opus only.	64 (older saved settings may show 32)
Opus complexity	Opus encoder effort, 0 to 10.	5
Constant bitrate (CBR)	Opus only: constant instead of variable bitrate.	off
In-band FEC	Opus only: lets the receiver recover single lost packets.	off
Expected loss (%)	Hint to the Opus FEC.	5
Opus bandwidth	Highest audio bandwidth Opus may use: narrowband (4 kHz), wideband (8 kHz), super-wide (12 kHz), full (20 kHz).	narrowband
Opus signal	Hint to the encoder: `music` favours waveform accuracy, `voice`, or `auto`.	music

Control	Meaning	Default and range
Jitter buffer (ms)	Playout buffer on this end. Larger absorbs uneven arrival but adds delay.	80
Playback channels	2 copies the audio to left and right; 1 plays mono.	2
Smooth packet timing	Sends one packet per frame interval on a strict clock. Keep it on.	on
Debug: save WAV files	Writes `bridge_play.wav` and `bridge_capture.wav` to your home folder for analysis with `wavstat.py`. Not remembered between runs.	off

The transmit codec affects only what this end sends. The server's setting therefore decides the quality of receive audio, and the client's setting decides the quality of transmit audio.

Start, Stop and link indicator

Start opens the audio devices and the network. If a saved device is missing it waits, as described under Audio devices. Stop ends the bridge, or cancels the wait. The indicator reads **Connected** (green) when packets are arriving, and amber while waiting for the other end.

Meters and gain sliders

There are two level meters, labelled by role. On the server they read "RX audio from radio" and "TX audio to radio". On the client they read "TX audio from FreeDV" and "RX audio to FreeDV". Each has a gain slider beside it, from -40 to +12 dB in 0.5 dB steps, and a "0 dB" reset button. The meters show the level after the gain is applied, on a scale from -60 dB to full scale. Positive gain can clip, so watch the meter.

Status lines and log

Two lines under the meters show the counters explained in Reading the status display. The log below them records connections, warnings, and the version line. Its timestamps help when matching events between the two ends.

Recommended settings

Start with PCM at 48000 Hz and 10 ms frames, with a jitter buffer of 110 ms. PCM keeps the radio's waveform exact, which FreeDV needs, and costs about 780 kbps each way, which a wired LAN or Tailscale handles easily.

PCM or Opus

Opus uses far less bandwidth (about 65 kbps at 64 kbps) but is built to preserve how audio sounds, not the exact waveform. On a modem-like test signal (many carriers between 400 and 2800 Hz, not a real FreeDV signal) the coding noise Opus adds was:

Opus settings	Fidelity, dB (higher is better)
48 kHz, 32 kbps, default	9
48 kHz, 32 kbps, narrowband and music	17
48 kHz, 64 kbps, default	16
48 kHz, 64 kbps, narrowband and music (current default)	27.5
8 kHz sampling, 64 kbps (not offered: device rates must stay at 48 kHz)	38

The narrowband and music settings are the defaults for that reason. A comparison of Opus and PCM on live FreeDV signals has not yet been made, and PCM is expected to decode better. Use Opus only where bandwidth matters, and try 64 kbps or more.

Jitter buffer

The default is 80 ms. Raise it to 110 to 200 ms on Wi-Fi or if Underruns climb. To find the smallest stable value, lower it in steps and watch Underruns for a minute or two. Stop at the lowest value where Underruns stays at 0. The server's buffer handles the audio coming from the client, and the client's buffer handles audio from the server, so tune each end.

Transmit level

For digital modes, the aim is full power with the radio's ALC meter idle. The ALC should not move. Power output is the reliable indicator, because a flat ALC does not mean the audio is missing. Lower the server's "TX audio to radio" slider until power matches the radio's

setting. This installation uses -9 dB. FreeDV's own transmit level on the client also changes the drive.

Frame size and rate

Keep 48000 Hz and 10 ms unless you have a reason. Both ends must match, and a mismatch shows as a rising Bad count. A frame must fit in one UDP packet, so PCM at 48000 Hz accepts only 5 and 10 ms frames; 20 ms and longer need Opus or a lower sample rate.

Reading the status display

Read the counters on the **receiving** end of a direction: the server's Lost figure counts packets missing between the client and the server, and the client's counts the other way. On this installation over Tailscale, loss settled at about 0.2 to 0.3%, with no packets late, failed or dropped.

Counter	What it counts	Healthy	If it is not
TX kbps (pkts)	Data rate sent, and the number of packets sent.	PCM about 780 kbps. Opus about 7 kbps when silent, 35 to 70 with audio.	0 means this end is not sending (no capture audio, or link down).
RX packets	Audio packets received.	Rises by about 100 a second.	Not rising: check the link indicator and the other end.
Lost (events, longest)	Packets missing from the sequence, how many separate gaps, and the longest gap in packets.	Under 0.5% of RX packets, with short gaps.	See Troubleshooting: high loss.
Late	Packets that arrived after a later one.	0	Reordering on the path.
TX errors	Packets this end failed to send.	0	A local network problem; the log shows the error text.

Counter	What it counts	Healthy	If it is not
Bad	Packets that failed to decode or had the wrong size.	0	Sample rate or frame size differs between the ends, or Opus is unavailable on one end.
Underruns	Times playback ran out of audio.	A few per hour at most.	Raise the jitter buffer.
Dropped	Audio discarded because the buffer was full.	0	Raise the jitter buffer; check for CPU load.
Buffer	Frames waiting in the jitter buffer.	Near the jitter setting divided by the frame size.	Persistently 0: underruns; persistently high: delay.

Send timing

The second line shows how evenly this end sends packets: the number sent too close to the previous one (under half a frame), the number too late (over one and a half frames), and the longest gap since the last refresh.

- **Too-close** should be near 0. Packets sent back to back were dropped by the path out of a Mac (about 20% loss for pairs), so this matters.
- **Too-late** is normally about 6% of sends, with the longest gap at 20 to 30 ms. That is harmless: the receiver's buffer absorbs it.

Link indicator and log

The server shows "Connected - Client address" when packets arrive, and "Waiting for client..." otherwise. The client shows "Connected - Server" or "No data from server". The log records `Link up` and `Link lost`. The link counts as lost after 5 seconds without packets.

Level meters

Use them to confirm audio is flowing. A meter that stays flat while the other end is active points to the wrong device, a muted device, or (on a Mac) a missing microphone permission.

Command-line use

The bridge also runs without a window, which suits a server left running in a terminal. The command-line version does not read the window's settings file; give it every option you need.

```
python3 audio-bridge.py list
python3 audio-bridge.py server --wait --in-dev "<capture device>" --out-dev "
<playback device>" --bind <server address>
python3 audio-bridge.py client --host <server address> --in-dev "<cable B>" -
-out-dev "<cable A>"
```

`list` prints the audio devices. A device is given by its number or by part of its name. While running, the program prints a statistics line every 5 seconds with the same counters as the window, and stops on Ctrl-C.

Waiting for the radio

With `--wait`, the program keeps retrying every 3 seconds until the audio devices exist, so you can start it before switching the radio on. Without it, a missing device ends the program at once with an error naming the device. From version 2026-10-25a, --wait also covers the radio being switched off while the bridge runs: the bridge notices that its audio has stopped, waits, and starts again by itself when the devices return. Older versions needed a restart after the radio came back.

Running the bridge in the background on the server

On the radio's computer, a systemd user service can start the bridge at boot and keep it running. The file `audio-bridge.service` holds the command, with comments on the three things to change. The command it runs is:

```
python3 audio-bridge.py server --bind YOUR-TAILSCALE-ADDRESS --port 5800 \
    --in-dev alsa_input.usb-Burr-Brown --out-dev alsa_output.usb-Burr-Brown \
    --rate 48000 --frame-ms 10 --jitter-ms 110 --out-gain-db -9 --codec pcm -
-wait --pin-volumes 100
```

- `server` runs the radio's end. `--bind` is the computer's Tailscale address, so only the tailnet can reach it (the bridge has no password). `--port` is 5800.
- `--in-dev` is the radio's capture device (the audio coming from the radio), and `--out-dev` is its playback device (the audio going to the radio). Each takes a number or a name substring. A name is safer, because the numbers can change between boots. List

the devices with `python3 audio-bridge.py list` and use a substring that matches only one.

- `--rate` and `--frame-ms` must match the panel (48000 and 10 by default). `--jitter-ms` is the buffer for audio arriving from the panel, and 110 suits Wi-Fi.
- `--out-gain-db -9` sets the level sent to the radio, which is the transmit drive. Adjust it while watching the radio's ALC.
- `--codec` is what the server transmits: `pcm`, or `opus` with `--bitrate 64000 --opus-bw nb --opus-signal music`. The panel's own codec setting is separate.
- `--wait` starts the bridge before the radio is on, and lets it recover when the radio is switched off and on again.

To install and start it:

```
mkdir -p ~/.config/systemd/user
cp audio-bridge.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now audio-bridge
loginctl enable-linger "$USER"
journalctl --user -u audio-bridge -f
```

It is a user service, not a system one, because the radio's sound devices belong to your own login session (PipeWire or PulseAudio). The `loginctl` line keeps it running when you are not logged in. Close the bridge window, and any other copy of the bridge, before starting it, because only one program can use UDP port 5800. After editing the file, run `systemctl --user daemon-reload` and `systemctl --user restart audio-bridge`. If the Tailscale address is not up yet at boot, the bridge cannot bind, exits, and the service starts it again every 5 seconds until it can.

Option	Meaning	Default
`--host`	Client: server address.	none
`--port`	UDP port.	5800
`--bind`	Server: address to listen on.	0.0.0.0
`--in-dev`, `--out-dev`	Capture and playback devices.	system default
`--rate`	Sample rate; same on both ends.	48000

Option	Meaning	Default
`--frame-ms`	Frame size; same on both ends.	10
`--jitter-ms`	Playout buffer.	80
`--codec`	`pcm` or `opus`, for what this end sends.	pcm
`--bitrate`	Opus bitrate in bit/s.	64000
`--opus-bw`	`nb`, `wb`, `swb` or `fb`.	nb
`--opus-signal`	`music`, `voice` or `auto`.	music
`--complexity`	Opus effort, 0 to 10.	5
`--cbr`, `--fec`, `--loss`	Opus constant bitrate, in-band FEC, expected loss percent.	off, off, 5
`--out-channels`	2 copies audio to both channels; 1 is mono.	2
`--in-gain-db`, `--out-gain-db`	Capture and playback gain in dB.	0
`--no-pace`	Turns off the strict send clock. Not recommended.	pacing on
`--wait`	Retry until the audio devices exist.	off
`--dump-play`, `--dump-capture`	Record played or captured audio to a WAV file, for debugging.	off

Programs started from a terminal stop when the terminal closes. Use `tmux`, `screen` or a service to keep the bridge running after you log out.

Network and security

Neither the bridge nor rigctld has any authentication or encryption, so run both over a private network such as Tailscale, or a firewall that admits only your own machines. Anyone who can reach the bridge's UDP port can inject audio into the radio's transmit input, and anyone who can reach rigctld can key the transmitter.

Service	Port	Direction	Notes
Audio bridge	UDP 5800	Both ways	The client starts the conversation and sends a keepalive every second, so only the server needs to be reachable.
rigctld	TCP 4532	Client to server	Plain text, no password.

Limiting exposure

- Set the server's Bind address to its Tailscale address, so the bridge listens only on that network.
- Start rigctld with `-T <server Tailscale address>`. Its default is to listen on **all** addresses.
- On the client, use the server's Tailscale address in Server host/IP, and in the panel and FreeDV.
- Allow the first-run firewall prompt for Python on the server. Windows may class the Tailscale interface as a public network.

Network quality

- Prefer a wired connection or 5 GHz Wi-Fi close to the access point. On a Mac, switching off AirDrop and Handoff removes a source of periodic Wi-Fi delay spikes.
- Tailscale's packet size limit is 1280 bytes. PCM packets at 48000 Hz and 10 ms are about 970 bytes and fit. The bridge refuses PCM frame sizes that would not fit in one packet.
- Check the path with `tailscale ping <address>` (direct paths are best), or with the loss tester described under Diagnostic tools.

Why packets are sent on a strict clock

Audio callbacks deliver frames unevenly, and packets sent back to back were being dropped on the way out of the Mac: about 20% of pairs, and 3% even when loosely spaced. Sending exactly one packet every 10 ms brought loss on this installation down to about 0.2 to 0.3%. The "Smooth packet timing" option does this and should stay on.

Operating safely

- Test new settings into a dummy load at low power.
- Remote operation of a transmitter may carry conditions under your licence. Check them.

- The radio's own transmit time-out (menu 036 TX TOT, from off up to 30 minutes) is the last protection if a link fails while it is transmitting. Keep it enabled.
- The tuner transmits a carrier, and Tune starts at once, with no confirmation. Connect an antenna or dummy load first.

Radio control with Hamlib

Hamlib's `rigctld` is the one program that owns the radio's serial (CAT) port. FreeDV, the panel and a logging program all talk to rigctld over the network, and it takes their commands one at a time and gives each program only its own replies. This replaced the earlier setup in which `ser2net` shared the raw serial line, where every program saw every reply.

Radio settings (FT-991A)

- Menu 031, CAT RATE: set to **38400**, the fastest of the four choices (4800, 9600, 19200, 38400). It must match rigctld's `--serial-speed`.
- Menu 032, CAT TOT: the time the radio waits for the rest of a command. If you see odd protocol errors, try a longer value.

The radio's USB connection provides two serial ports. Only one carries CAT; on this installation it is the `/dev/yaesu-cat` shortcut, which points to `/dev/ttyUSB0`. If you are unsure which port answers, test it with the CAT command `FA;`, which should reply with the frequency.

Starting rigctld

```
rigctld --model=1035 --rig-file=/dev/yaesu-cat --port=4532 --serial-
speed=38400 -T <server Tailscale address>
```

Model 1035 is the FT-991. Add `-vvv` to print what rigctld is doing. Only one program can hold the serial port, so stop `ser2net` first. A rigctld that was started but cannot talk to the radio may sit without listening; run it in the foreground with `-vvv` to see why.

Testing it

Ask for the frequency over a plain connection. A number such as `14074055` means rigctld and the radio both work:

```
printf 'f\n' | nc -w 2 <server Tailscale address> 4532
```

For a full report (frequency, mode, PTT, power setting, meters, tuner, capabilities), run `bash rigtest.sh` on the server. It reads values only and never transmits.

What the radio exposes through Hamlib (this installation)

Item	Value
Power setting	`RFPOWER`, 0.05 to 1.0, as a fraction of 100 W
Power output	`RFPOWER_METER_WATTS`, 0 to 100 W, calibrated by Hamlib
SWR	`SWR`, 0 to 5, as a ratio
ALC	`ALC`, 0 to 1
Tuner	`TUNER` function; `TUNE` operation
Data modes	`PKTUSB` and `PKTLSB` are the radio's DATA-USB and DATA-LSB

Starting it at boot

A service definition for rigctld has not been set up. Until then, start it by hand, or detach it from your terminal (for example with `setsid`) so that closing the window does not stop it.

The FT-991A panel

The panel shows the radio's VFO A and VFO B frequencies, mode, S meter, power output, SWR and ALC on the operating computer, and sets frequency, mode, bandwidth, AGC, power, the antenna tuner and (with Raw CAT) split, a VFO swap and the radio's power. It also has a push-to-talk button and the Mic level and Volume sliders. It needs only Python and tkinter. Run it with:

```
python3 ft991a-panel.py --host <server Tailscale address> --port 4532
```

With no options it opens a window where you type the host and port and press Connect. It remembers them, and its size and position. The layout is narrow, about 470 pixels wide, so it fits beside other windows. The window is as tall as the tab that is showing, so a short tab leaves no empty space under it. The version number is shown in the title bar, and --version prints it. The window size is saved under a newer settings key, so an older, wider saved size is ignored.

Starting up. When the program opens it connects to the radio at the last server you used, and then starts the audio there too, if you have set the audio up before. Neither shows a dialog if it cannot: the log says why, and the **Connect** and **Start audio** buttons work as always. If the radio is switched off, the connection still comes up, and **Power on** can start the radio. To turn this off, set `"auto_connect": false` in `~/.ft991a-panel.json`.

Log. The **Hide log** button above the log hides it, and the window shrinks to fit. The button then reads **Show log**. The log keeps collecting messages while it is hidden, and Copy report on the Audio tab still includes them. The choice is remembered (`log_hidden` in the settings file).

Connection types

The panel can reach the radio in two ways. Choose one in the Connection dropdown, or with `--backend hamlib` or `--backend cat`. The type cannot be changed while connected, and each type remembers its own port and its own correction points.

Connection	Port	Meters	Notes
Hamlib rigctld (default)	4532	Watts and an SWR ratio, calibrated by Hamlib	Orderly: rigctld gives each program only its own replies.
Raw CAT (ser2net)	The ser2net port for the radio's serial line, which must be in raw mode	Raw 0 to 255 values, shown as "uncalibrated" until you use Cal PO and Cal SWR	Every program on the port sees every reply.

Raw CAT sends the radio's own commands: `TX;`, `IF;`, `PC;` and `AC;` about every second, and `RM5;`, `RM6;` and `RM4;` several times a second while transmitting. Because ser2net shows each program every reply, the panel's polling also appears in FreeDV's and the logger's streams, which a Hamlib client may not tolerate. Test with FreeDV idle first. In this mode Stop tune uses the radio's own tuning toggle (`AC002;`) instead of PTT off. The raw connection also avoids the Hamlib client problem described under Known issues.

Display

Item	Meaning
VFO A and VFO B	Two boxes side by side, each showing a frequency such as `14.074.055`. VFO A, the operating VFO, has a light blue background. Click VFO B to swap the two VFOs (Raw CAT only; see Swapping the VFOs below). With the Hamlib connection only one frequency can be read, shown under VFO A, and VFO B shows dashes.
RX, TX, TUNING	Green RX when receiving, red TX when the radio is keyed (by FreeDV or anything else), amber TUNING during a tuning cycle.
S meter	While receiving: S units, amber above S9. With Raw CAT the raw reading is shown in brackets, and a ~ in front of the reading means the scale is still approximate until you press Cal S9. With Hamlib the reading comes from Hamlib's own scale. It shows dashes while transmitting.
Power out	In watts, with a peak marker that holds for 2.5 seconds. Live only while transmitting.
SWR	As a ratio, on a bar from 1:1 to 5:1. Amber from 2:1, red from 3:1.
ALC	As a percentage of the radio's own ALC meter, which reaches mid-scale at 50% (the panel doubles what CAT reports, to match the radio). Green up to 25%, orange up to 50%, red above 50%.
Status	Connected, No reply from radio, Connecting, or Not connected. Values grey out when replies stop for 3 seconds.

Audio in and **Mic** show the level of the audio coming from the radio and of your microphone, the same two level meters as on the Audio tab. They sit at zero while the audio is stopped. All the meters are about half their earlier height.

Controls

- **Mode:** DATA-USB (Hamlib's `PKTUSB`, used by FreeDV), USB, LSB, DATA-LSB, CW-U, CW-L, AM, AM-N, FM, FM-N, DATA-FM, RTTY-LSB, RTTY-USB, C4FM. The mode is set with Hamlib's default passband.
- **Power:** 5, 25, 50, 75 or 100 W. On 144 and 430 MHz the menu shows 5, 25 and 50 W.
- **Tuner:** the ATU button puts the radio's internal antenna tuner in line (the light beside the button turns yellow) or bypasses it (the light turns green, and it flashes yellow during a tuning cycle), the same as a short press of the radio's TUNER key. It works with both connection types, and it is ignored while a tuning cycle runs. The radio's menu 141 TUNER SELECT must be set to INTERNAL.
- **Tune:** starts a tuning cycle at once. While tuning, the button reads Stop tune.

Split

The FT-991A's CAT command list has no command to choose the active VFO and none dedicated to split. It has `FT`, which sets which VFO transmits, and on this radio that is how split works. **Split off** means VFO A transmits (`FT2;`). **Split on** means receive on VFO A and transmit on VFO B (`FT3;`). The **Split** button beside the VFO boxes toggles it, reads "Split: ON" or "Split: OFF", and follows the radio if split is changed on its front panel. Clicking a VFO box does not change split.

The light blue box is always VFO A, the operating VFO, because CAT cannot operate on B. The captions say what each VFO does: with split off, A shows "(RX/TX)"; with split on, A shows "(RX)" and B shows "(TX)". CAT cannot set the radio to receive on B and transmit on A directly, but swapping the VFOs while split is on does exchange the two frequencies. Split changes and swaps are refused while the radio is transmitting. With split on, the radio transmits on VFO B's frequency, so check it, and test into a dummy load first. With the Hamlib connection the button uses rigctld's split commands (`S` and `s`) and VFO B is not shown.

Swapping the VFOs

CAT cannot make VFO B the operating VFO, so the panel uses the `SV` command instead. It swaps the contents of VFO A and VFO B, both frequency and mode. Click the **VFO B** box: B's frequency moves into VFO A, which is the one the radio operates on, and the old A frequency moves into B. Click VFO B again to swap back. Clicking VFO A does nothing. The panel follows swaps made on the radio itself. Swaps are refused while the radio is transmitting. Only VFO B shows a hand cursor, as it is the only box that responds to a click.

Changing the frequency

There are two ways. Both work on VFO A with either connection, and on VFO B with Raw CAT only, and both are refused while the radio is transmitting. The radio tunes from 30 kHz to 470 MHz.

- **Typed:** type into the **Go to** box and press Enter to set VFO A, or press **Set A** or **Set B**. Write MHz (`14.074`), kHz (`7074k` or `14074`) or Hz (`14074000`), with an optional unit (`145.5 MHz`) or grouped digits (`14.074.000`). A bare number under 1000 is MHz, up to 470000 is kHz, and anything larger is Hz.
- **Digit by digit:** each frequency is nine digits, from 100 MHz down to 1 Hz. The 100 MHz digit is there so 144 and 430 MHz can be tuned. Click a digit to select it (it turns amber), then use the mouse wheel to change that digit. With the readout focused, Up and Down change the selected digit, and Left and Right move the selection along the digits. The frequency stops at the radio's limits. The area around the digits still behaves as before: clicking around VFO B swaps the VFOs, but clicking a digit never does.

Band. The **Band** pull-down to the left of the Go to box shows the band that VFO A is on. It reads "Band" when the frequency is outside the amateur bands (160, 80, 60, 40, 30, 20, 17, 15, 12, 10 and 6 m, 2 m and 70 cm). Pick a band to go to the last frequency and mode you used on it. The first time, it goes to a usual starting point: 1.900 LSB on 160 m, 3.800 LSB on 80 m, 5.3305 USB on 60 m, 7.200 LSB on 40 m, 10.130 USB on 30 m, then 14.200, 18.130, 21.300, 24.950, 28.400 and 50.125 USB, 146.520 FM on 2 m and 446.000 FM on 70 cm. The panel remembers the last frequency and mode on each band between runs (as `band_memory` in the settings file). Picking a band does what Set A does: it changes VFO A, it needs a connection, and it is refused with a message while the radio is transmitting. The mode is sent after the frequency.

A burst of wheel clicks is sent to the radio as one command, and the display shows your value until the radio confirms it.

C4FM and FM

The **C4FM** button switches between FM and C4FM, and reads "C4FM: ON" or "C4FM: OFF". It is available on VHF and UHF (VFO A at 100 MHz or higher) and greyed out on HF. Pressing it when the mode is neither C4FM nor FM selects C4FM. The Mode menu also lists C4FM, FM, FM-N and DATA-FM. How the radio transmits in C4FM depends on its menu 090 AMS TX MODE, which the panel does not change.

Bandwidth

There is one **Bandwidth** slider, not one per VFO, because the CAT width command (`SH`) has no VFO parameter. It sets the receive width of the operating VFO, A. It works in SSB, CW, RTTY and data modes and is greyed out with "fixed in this mode" in AM, FM and C4FM. The slider covers the radio's narrow and wide width sets together and switches the radio's NARROW state for you when a width needs it. In the DATA modes the width table follows menu 062 DATA MODE: PSK uses the RTTY/PSK widths, and OTHER is assumed to use the SSB widths. The slider needs the Raw CAT connection.

The Audio tab

The **Audio** tab, beside the Radio tab, routes four audio endpoints, as FreeDV's audio settings do: **Radio to computer**, **Computer to speaker**, **Mic to computer** and **Computer to radio**. The audio engine is built into the panel. It is the client side of the audio bridge, which keeps running on the server and speaks the same protocol, so no second program or file is needed on this computer. Only the `sounddevice` package is required, plus `opuslib` if you want Opus. The tab never changes the radio's own volume or menu settings. Without sounddevice it says so, and the rest of the panel works as before.

- **Radio:** choose **Remote (server)** for a radio on the server, reached through the bridge, or **Local (sound card)** for a radio connected to this computer.
- **Server and UDP** (remote only): the audio server's address and port (5800). A blank server means the radio tab's host.
- **Receive: Radio to computer** is where the radio's audio comes from, and **Computer to speaker** is where you hear it. The Test button beside it plays a tone on that device. **Volume** runs from 0 to 200%, where 100% leaves the audio unchanged, and a level meter sits below it. Volume is a software level on this computer, so it changes only what you hear.
- **Transmit: Mic to computer** is the microphone that supplies the transmit audio, and **Computer to radio** is where that audio goes. With a remote radio your microphone goes to the bridge directly, and the cable named here is read as well, so a program such as FreeDV can send its transmit audio through it. **Mic level** runs from 0 to 200% and sets how loud your voice is sent, which is the transmit drive from the computer, with a level meter below it. The **Mic** button switches the microphone on and off. It always starts OFF (muted) when audio starts, and the microphone is sent only while the button reads ON or while PTT is on. Choose "(none - listen only)" as the microphone to listen without sending any microphone audio. **Rescan** refreshes the device lists. Your choices are saved, and a saved device that is missing stays visible as "(not present)" instead of being replaced.

- **Codec and Jitter** (remote only): `pcm` or, if opuslib is installed, `opus`. The jitter buffer starts at 110 ms. Opus at about 64 kbps suits voice and uses far less bandwidth. Use PCM for FreeDV's digital voice and other modem-like signals, which Opus can distort.
- **Advanced** shows the bridge's own options. **Rate** and **Frame** default to 48000 Hz and 10 ms, and PCM at 48000 Hz needs frames of 10 ms or less, so use Opus for longer frames. **Smooth packet timing** (remote only) spaces the packets evenly, which stops bursts being dropped on Wi-Fi, so leave it on. **Stereo playback** plays the audio on both channels, and should stay on unless the device is mono-only. For Opus there are the bitrate in kbps, the bandwidth, the signal type, and optional loss protection (FEC) with the expected loss. The server must use the same codec, rate and frame size; a mismatch shows up as "bad" packets in the statistics. Debug WAV recording, Opus complexity and constant bitrate stay in the bridge window and its command line.
- **Start audio** and **Stop audio** start and stop the audio. Every setting is locked while it runs.
- The status text at the bottom shows the bridge's statistics for a remote radio: whether the server is connected, packets received, lost (with loss events and the longest gap), late and bad, underruns, dropped packets and the buffer level, the send rate and errors, and the send timing. It also shows underruns and dropped audio for the receive and transmit paths. The log records when the audio link comes up and is lost.

What each dropdown offers. Each dropdown lists only the devices that suit its role. **Computer to speaker** and **Mic to computer** list real devices, never virtual cables such as BlackHole. With a remote radio, **Radio to computer** and **Computer to radio** list "(direct)" and the virtual cables, because the bridge plays into one cable and reads from the other: Radio to computer lists the cables it can play into, and Computer to radio lists the cables it can record from. With a local radio they list real sound devices, such as the radio's USB codec. Tick **Show all** to list every device in every box. If an older settings file had a virtual cable as your speakers or microphone, the panel replaces it with a default and says so in the log. It also refuses to start if a cable is chosen as the speakers or microphone, or if the same cable is used for both radio ends.

Setting it up

Local radio. The radio's USB codec is on this computer. Set Radio to Local, **Radio to computer** to the radio's input (the USB audio codec), **Computer to speaker** to your speakers, **Mic to computer** to your microphone and **Computer to radio** to the radio's output (the same USB audio codec). Nothing goes over the network.

Remote radio. The radio is on the server, which runs the bridge. Set Radio to Remote and enter the server's address. Each radio end offers "(direct)" and the virtual cables on this computer:

- On a MacBook with BlackHole, set **Radio to computer** to BlackHole 2ch and **Computer to radio** to BlackHole 16ch. The panel's built-in bridge plays the radio's audio into BlackHole 2ch, for FreeDV, and also sends a copy straight to **Computer to speaker** (for example the MacBook Air speakers), without reading the cable back. Your microphone (**Mic to computer**, for example the MacBook Air microphone) goes straight to the bridge in memory, so nothing of yours is written into BlackHole 16ch. The bridge also reads BlackHole 16ch, so whatever FreeDV writes there is sent to the radio too, mixed with your microphone while Mic or PTT is on. FreeDV can use the same two cables: Input From Radio is BlackHole 2ch and Output To Radio is BlackHole 16ch.
- When FreeDV supplies the transmit audio, set Mic to computer to "(none - listen only)". The bridge still sends whatever FreeDV writes to the cable.
- Choose "(direct)" for a radio end to skip the cable. The network audio then plays straight on the speakers, or the microphone goes straight to the network. That has the least delay and needs no virtual cable, but nothing else can share the audio.

If you hear nothing

The status text at the bottom of the Audio tab shows the receive level at each stage over the last second, with a hint about the first silent stage. Use it to find where the audio stops:

- **"from the network" is 0%:** the server is sending silence. Check the radio's squelch, mode and receive audio, and the input device the bridge uses on the server. Also close any other bridge window, because the server sends audio to one client at a time.
- **"from the radio's sound card" is 0% (local radio):** nothing arrives from the radio's input device. Check the device chosen for Radio to computer and the radio's own audio settings. On a Mac, check that Terminal or Python may use the microphone (System Settings, Privacy and Security, Microphone), because macOS treats any input device as a microphone.
- **"to the speakers" is 0% but the stages before it are not:** check Volume, which may be at 0%, and the speaker device. Press **Test** beside Computer to speaker, which plays a 1 kHz tone on that device whether or not audio is running. If you cannot hear the tone, the problem is the device or the Mac's output volume, not the routing.
- Warnings and errors from the sound devices appear in the log, for example "receive path: the input device reported: ...".

For transmit, the status text also shows the **microphone** level and what is **sent to the server**. If the microphone reads 0% while Mic is ON, check the microphone, its level in the system sound settings, and that this program may use the microphone. If the microphone shows a level but nothing is sent to the server, the audio is not leaving the panel: check the server address, and that the bridge window is closed, because the server sends and receives with one client at a time. Your microphone no longer appears on the BlackHole 16ch cable, because it goes to the bridge directly; only FreeDV's audio does.

Copy report. The **Copy report** button on the Audio tab puts a plain-text report on the clipboard and also saves it as `ft991a-panel-report.txt` in your home folder. It holds the panel and engine versions, the Python and sound library in use, the audio settings and devices, the state of the radio connection, the status text, and the last lines of the log. IPv4 addresses are hidden, except 127.0.0.1. Paste it into a bug report; it is the quickest way to show what the panel was doing.

When your audio reaches the server but not the radio. The transmit line also shows the radio's **ALC** while you transmit. In USB, LSB and the data modes, the ALC moves when audio reaches the radio's modulator. If the microphone is live and audio is being sent, but the ALC stays flat, the panel says so: the audio is not reaching the radio, or is far too quiet. The usual cause is on the radio's computer: the sound card's volume turned down or muted, the bridge writing to the wrong output device, or the bridge's own gain (`--out-gain-db`). The other cause is the radio itself. On the FT-991A the source of the transmit audio is set per mode in menus, for example 070 DATA IN SELECT and 072 DATA PORT SELECT for the data modes, 106 SSB MIC SELECT and 109 SSB PORT SELECT for SSB, 045 and 048 for AM, and 074 for FM. If a mode is set to take its audio from the microphone or the DATA jack, the radio ignores the USB audio completely.

Sound settings that seem to change by themselves. On Linux, volumes are remembered per sound card and per program, and the radio's sound card disappears when the radio is switched off and appears again when it is switched on, so a level can come back different. FreeDV and other programs can also change levels. To see what is happening on the radio's computer, run `pactl list sinks` and `pactl list sink-inputs` and read the Volume and Mute lines for the radio's sound card and for the bridge. To catch the moment something changes, run `pactl subscribe` in a terminal and then switch the radio off and on, or start and stop FreeDV: it prints a line each time a sound card or stream changes. A volume can be put back with `pactl set-sink-volume NAME 100%`, using the name from the list.

Keeping the volumes up. The bridge can keep the volume of the radio's sound card, and of its own streams, where you want it. Add `--pin-volumes 100` to the command (the service file has it), or tick "Keep the radio's sound volumes at 100%" in the bridge window. Every `--pin-interval` seconds (10 by default) the bridge reads the PulseAudio or PipeWire volumes of the radio's playback and capture devices and of its own two streams, and puts back any that are lower than that percentage or muted. It leaves other programs' streams alone, so FreeDV's own volume is not touched. Each correction is printed in the bridge's log (the journal, for the service), for example "the radio's playback volume was 35%, set to 100%", which also shows when something turns the volume down. It uses the `pactl` command and does nothing, with a note, if the radio's devices are not PulseAudio devices or `pactl` is missing. What you see change in a mixer such as pavucontrol is this layer, not the radio's own settings.

Noise reduction

The **Radio** tab has a **Noise reduction** check box, a **Level** pull-down from 1 to 20 and an **Auto-calibrate** button, below the ATU row and the sliders and above the meters. Noise reduction needs numpy (`pip install numpy`) and the sound library, because it works on the audio that goes to your speakers. Without numpy the rest of the panel works and these controls are greyed out, and the line under them says why. The Mac and Windows installers include numpy, which makes them roughly 20 MB larger.

- **What it is.** Spectral subtraction. The audio is cut into 20 ms windows that overlap by half. The panel keeps a picture of what the noise looks like at each frequency and subtracts it from every window. It never cuts a frequency right down to nothing (there is a floor), because that is what causes the "musical" twittering that simple noise reduction is known for, and it smooths the power over neighbouring frequencies and over time for the same reason. It uses about 1% of one processor core. It delays the audio by about 20 ms whether it is on or off, so switching it never clicks.
- **Where it works.** Only on the audio that goes to your speakers: with a local radio, with a remote radio through the cable, and with a remote radio set to (direct). The cable that FreeDV reads is never touched, because noise reduction damages a digital voice signal. It is meant for listening to voice, so leave it off for digital modes.
- **Level.** 1 is gentle and 20 is strong. On steady receiver noise it takes out about 6 dB at level 1, 10 dB at level 5, 16 dB at level 10, 22 dB at level 15 and 27 dB at level 20 (measured on simulated noise after calibrating). A steady tone or whistle is reduced by the same amount as the hiss. Voices come out best at levels 6 to 14. Higher levels remove more noise but begin to thin the voice. Changing the level re-tunes the subtraction at once, without calibrating again.

- **Without calibration.** It follows the noise by itself, using the quietest power it has seen over the last second or two. That removes about 13 dB at level 8, but it takes a little more out of speech than a calibrated profile does.

Auto-calibrate teaches it what the noise on your band sounds like. It needs the audio running and the radio connected (either connection), and the radio not transmitting.

1. It tunes VFO A to a frequency beside yours, nearest first. For SSB, CW, RTTY and the data modes it tries 3.75, 6.25 and 8.75 kHz above and below you. Stations sit on round frequencies, so these are 1.25 kHz away from the busy whole and half kilohertz. They are counted on the absolute frequency grid, so they end in .25 or .75 kHz wherever you are tuned. None is nearer than 3.5 kHz, because closer in, the signal you are listening to would still be inside the audio passband and would be learned as noise. For AM and AM-N it tries +10, -10, +15, -15, +20 and -20 kHz. For FM, FM-N and DATA-FM it tries +25, -25, +50, -50, +75 and -75 kHz, and the squelch must be open. It does not apply to C4FM.
2. At each one it waits 1.2 seconds for the retune to settle, then listens for 1.5 seconds. A frequency counts as noise only if its audio has no narrow line (a carrier, a whistle, a tone, a voice) and its power is steady (no speech, no static crashes, no fading). The log gives each frequency's strongest line and its pitch. A line that stays at the same pitch on three or more different frequencies cannot be a signal, because a signal changes pitch when the radio is retuned: it comes from the receiver or the audio path (a spur, a tone, hum). The panel then ignores that line when it judges the frequencies, says so in the log, and picks the nearest frequency that is otherwise clear. The line stays in the profile, so noise reduction takes it out like the rest. If no frequency qualifies even then, it uses the least bad one and says so.
3. It measures four seconds of noise there, builds the noise profile, and tunes the subtraction to the level you chose and to how steady the noise was. Noise that is not steady makes it more cautious, because subtracting a noise picture that keeps changing is what produces artefacts.
4. It retunes to your frequency, turns the speakers back on, switches noise reduction on, and reports what it found, for example "calibrated at +5 kHz: noise -38 dBFS, steady".

The whole thing takes about 10 to 20 seconds. Press the button again, which reads **Cancel** while it runs, to stop it.

The radio is away from your frequency while it runs, so it is careful. The speakers are silent, and PTT and Tune are refused until the radio is back. If anything unexpected happens it puts the radio back on your frequency first: the audio stops, the mode

changes, something else (FreeDV, VOX) keys the radio, there is no sound on any frequency (a closed FM squelch, say), or you close the panel. If somebody else changes the frequency, or the connection is lost, it ends without touching the radio, because putting it back would then be the wrong thing to do.

Calibrate again when the band, the antenna, the time of day or the AGC setting changes. Once calibrated, the profile keeps following slow changes in the noise by itself, but only from frequencies that look like noise. Spectral subtraction removes steady noise: hiss, band noise, fans, and steady tones and whistles. It does not remove static crashes, or interference that sounds like speech.

The Function tab

The **Function** tab, between Audio and Menu, holds the radio's front-panel functions in five small pages. It needs the Raw CAT connection for the functions, because Hamlib cannot reach them, and it says so when you use Hamlib. Nothing that the Radio tab or the meters already show is repeated here (AGC, the antenna tuner, bandwidth, power, ALC, SWR), and the radio's own volume is never touched.

- **Receive.** Preamp (IPO, AMP 1 or AMP 2), attenuator, RF gain, squelch, contour and its frequency, and the audio peak filter (APF, which the radio offers in CW).
- **Noise.** Noise blanker and its level, the radio's own noise reduction (DNR) and its level, auto notch, and manual notch with its frequency. This DNR is in the radio. It is not the panel's own noise reduction, which is on the Radio tab and works on the audio sent to your speakers.
- **Transmit.** Speech processor and its level, mic equalizer, mic gain, monitor and its level, and VOX with its gain and delay. The VOX delay is the MIC or the DATA one, as menu 142 VOX SELECT says.
- **CW.** Keyer, break-in, keyer speed, sidetone pitch, break-in delay, the CW spot (the radio's own tone for tuning to a signal), and the Monitor level (MONI), which sets how loud the radio's own sidetone is: 0 silences it. Keying with a key or paddles, and sending text, is done in the CW window (see The CW window).
- **FM etc.** Repeater shift, tone mode, the CTCSS tone, and dial lock and fast step (both for VFO A). DCS codes and the clarifier are not on the tab.

How it works. The tab reads the functions on the page that is showing about every two seconds, so a change made with the radio's own controls appears here. A change you make is written at once and read back, and the control then shows what the radio kept. If the radio refuses a change, the control goes back and the log says so. A function that the

radio does not answer in its present mode is greyed out and says "not in this mode": for example the manual notch and contour in FM, the audio peak filter outside CW, and the repeater shift and tones outside FM. The mouse wheel never changes a setting.

Safeguards. Turning VOX on asks first, because VOX makes the radio transmit whenever sound reaches its microphone or data input, and that includes the panel's own audio and FreeDV. While a noise calibration is running, the receiver functions are refused, because they would change the noise being measured. After you change a receiver function, the log says that a calibration may no longer match, and pressing Auto-calibrate again gives the best result. The other functions can be changed while transmitting.

Tested on a real radio. The Function tab has been tried on a real FT-991A over ser2net, and every command was first checked against Yaesu's CAT manual. The Monitor level (MONI) slider on the CW page silences the radio's own sidetone. Try new settings on a dummy load. A control the radio does not answer in its present mode is greyed out.

The Menu tab

The **Menu** tab lists all 153 items of the radio's menu, as the radio's own menu does, and lets you change them. It needs the Raw CAT connection, because Hamlib has no way to reach the menu, and it says so when you use Hamlib. Unlike the Mic level and Volume sliders, which are software levels, this tab changes the radio's own settings, and they stay in the radio. (Before version 1.0.4 this tab was called Settings.)

- **Sections.** The 18 sections follow the menu's own order, and each shows its range of item numbers. Choosing one reads its items from the radio. **Read section** reads it again, and **Read all 153** reads everything in a few seconds. The status line says how many items have been read.
- **Controls.** An item that is OFF/ON or DISABLE/ENABLE is a check box. Any other list is a pull-down. A number is a slider with its value and unit beside it. The two very wide numbers, 017 CONTEST NUMBER and 151 PRESET FREQUENCY, are entry fields: type the number and press Return. 087 RADIO ID is read only. The mouse wheel scrolls the list and never changes a setting.
- **Changes.** A change is written at once and then read back, so the control always shows what the radio kept. If the radio refuses a value, the control goes back and the log says so. Nothing is written while the radio is transmitting, and a value outside an item's range is refused before it is sent.
- **Items that ask first.** 031 CAT RATE (the radio changes speed at once, and the panel and ser2net keep using the old one, so the connection stops working), 032 CAT TOT, 033 CAT RTS (the radio then only listens while the RTS line is set, which a connection

through ser2net does not control), 039 REF FREQ ADJ (it moves every frequency) and 149 EMERGENCY FREQ TX.

- **Where it matters for transmit audio.** The radio takes its transmit audio from the USB connection only if the menu for the mode says so: 070 DATA IN SELECT and 072 DATA PORT SELECT for the data modes, 106 SSB MIC SELECT and 109 SSB PORT SELECT for SSB, 045 and 048 for AM, and 074 for FM.

The table of items, with every list of choices and every range, was checked against Yaesu's CAT manual, and the manual has some slips that the panel works around. It prints 068 and 069 with their digit counts swapped, so the width of a value comes from the radio's own reply. It lists 100 RTTY SHIFT FREQ with the code 1 twice, so 170 Hz is taken to be 0. It gives 151 PRESET FREQUENCY as 00030000 to 47000000 with no unit, and the radio reaches 470 MHz, so one step is most likely 10 Hz. The panel shows that item as the plain number the radio uses, so check what it means on the radio's display before relying on it.

Using the panel on Windows

The panel is plain Python and has been tested on Windows (version 1.6.0): the network audio needed no extra software (choose "(direct)" for the radio ends), and once you have entered the server the panel reconnects by itself the next time it starts. One difference: the CW sidetone has noticeably more delay than on a Mac. It plays on the computer's default sound output through Windows' older MME audio interface, which adds delay. It affects only the tone you hear while keying; the keying itself is timed by the daemon on the radio's computer. You need Python 3 (the standard installer includes tkinter) and, for the Audio tab, the sounddevice package (`pip install sounddevice`), plus `pip install opuslib` if you want Opus. For Opus, put the 64-bit `opus.dll` in the same folder as `ft991a-panel.py` , which is where the panel looks for it. Start it with `python ft991a-panel.py` . Windows' Privacy settings must allow desktop apps to use the microphone.

Installers (no Python needed)

For macOS and Windows there are ready-to-run versions, built by GitHub's own machines, because a Mac app can only be built on a Mac and a Windows program only on Windows. Download them from the **Releases** page of the repository. The version number is in each file name, and the panel's title bar shows it too. Both kinds contain Python, Tk, numpy, the sound library and the Opus library, so nothing else needs installing.

- **macOS:** `ft-991a-apple-silicon-<version>.dmg` is for Macs with an Apple chip (M1 and later), and `ft-991a-intel-<version>.dmg` is for Intel Macs. To see which you have, choose Apple menu, then About This Mac. Open the disk image and drag FT-991A

Panel onto the Applications shortcut. The images are built on macOS 14 (Apple Silicon) and macOS 15 (Intel), and older versions of macOS may not run them.

- **Windows:** `FT-991A-Panel-Setup-<version>.exe` is an installer, with a Start menu entry, an uninstaller and an optional desktop shortcut, and it needs no administrator rights unless you install for all users. `FT-991A-Panel-<version>.exe` is the program on its own: copy it anywhere and run it. The panel has been tested on Windows (see Using the panel on Windows).

The first start. The apps are not signed, so the system objects the first time. On macOS 15 and later, try to open the app, then open System Settings, Privacy & Security, and click Open Anyway next to the message that FT-991A Panel was blocked. On macOS 14 and earlier, Control-click the app, choose Open, then Open. If macOS says the app is "damaged", run `xattr -dr com.apple.quarantine "/Applications/FT-991A Panel.app"` in Terminal once. The first time you start the audio, macOS asks whether the app may use the microphone: allow it. On Windows, SmartScreen may say "Windows protected your PC": click More info, then Run anyway.

Virtual cables are not included. BlackHole (macOS) and VB-Cable (Windows) are separate downloads, and you need them only to share the audio with FreeDV. Without them, choose "(direct)" for the radio ends of the Audio tab. On Windows, sharing with FreeDV takes two cables, such as VB-Cable A and B. The Windows installer also puts `opus.dll` beside the program, and a copy next to the program is used instead of the one built in, so you can replace it with a newer one.

Building a new version. In the repository, open Actions, choose Build installers, click Run workflow, and leave the Version box empty: the version is the one in `ft991a-panel.py`. The finished files are attached to a draft release, and publishing the draft puts them on the Releases page. Every build checks itself: the program starts and reports whether Tk, numpy, the sound library and Opus are really inside it, and the build stops with an error if the program is not the version in `ft991a-panel.py`. What changed in each version is listed in `CHANGELOG.md`.

Windows has no BlackHole, so use virtual cables from VB-Audio. Each cable appears as two one-way devices: a playback side named "... Input" and a record side named "... Output". The panel needs two cables, one for each direction, such as VB-Cable A and B. The names below are examples, so use whatever the dropdowns show:

- **Radio to computer:** the playback side of the first cable, for example "CABLE-A Input". The bridge plays the radio's audio into it.

- **Computer to radio:** the record side of the second cable, for example "CABLE-B Output". The bridge reads the transmit audio from it.
- **FreeDV** on the same computer uses the opposite sides: Input From Radio is the record side of the first cable ("CABLE-A Output"), and Output To Radio is the playback side of the second cable ("CABLE-B Input").
- **Computer to speaker** and **Mic to computer** stay your real speakers and microphone.

Windows lists each device once for every audio system (MME, DirectSound, WASAPI). Choose devices from the same system for all four boxes; if one fails to open, try MME.

One limit to know: the server sends audio to one client at a time, so close the bridge window before starting audio in the panel, and the other way round. Whichever connects later takes the stream. FreeDV keeps working while the panel's audio runs, because FreeDV talks to the virtual cables and the panel's bridge connects them to the server. To transmit, click **PTT** on the Radio tab (or press the space bar), or turn **Mic** ON and key the radio another way, such as VOX or another program.

The levels multiply: Mic level, the bridge server's TX gain and the radio's own audio level all affect the drive. Set the radio's level once and use Mic level for adjustment.

Push to talk, AGC, Mic level and Volume on the Radio tab

The **PTT** button, just above the log and so on every tab, keys the radio. Click it once to start transmitting and click it again to stop. The space bar does the same from anywhere in the window: one press on, and one press off. A space typed into a text box is left alone, holding the key down does not flip PTT back and forth, and the space bar no longer "presses" a button that has the keyboard focus. PTT sends the CAT command `TX1;` and stops with `TX0;` (with the Hamlib connection, `T 1` and `T 0`). While it is on, the button turns red and reads TRANSMITTING. When the audio is running with a microphone, the microphone is opened only while PTT is on, unless you had already turned **Mic** ON, and it is muted again when PTT goes off.

A toggled PTT stays on until you turn it off, and clicking another window does not stop it. It lets go of the radio by itself only after a time-out of 120 seconds (the `ptt_timeout` setting in the settings file, in seconds); when the connection drops, after which the radio's own TX time-out (menu 036) ends the transmission; and when you disconnect or close the panel. The space bar reaches the panel only while its window has the keyboard focus, so while you are working in another window, click the PTT button to stop. If you would rather PTT let go when the window loses focus, set `"ptt_release_on_focus_loss": true` in the settings file. PTT refuses to key while the antenna tuner is tuning. Use one keying source at a time, because FreeDV and the panel's PTT can conflict.

The **AGC** menu beside the Width slider sets the radio's AGC to Off, Fast, Mid, Slow or Auto (the `GT` command). It shows the radio's current setting and follows changes made on the radio.

Mic level and **Volume** on the Radio tab are the same controls as on the Audio tab, so moving one moves the other. They are software levels on this computer, so they never change the radio's own volume. Only the **Menu** tab changes the radio's menu.

Tuning

The tuner transmits a carrier while it tunes, so connect an antenna or dummy load and check the frequency is clear. The panel sends Hamlib's `TUNE` operation and watches the radio's PTT state. Pressing Stop tune sends PTT off. If the radio is still keyed 25 seconds after tuning started, the panel sends PTT off by itself. Because `TUNE` may not return until tuning ends, the panel waits up to 30 seconds for the reply, and other programs' commands may pause meanwhile. Tune starts without a confirmation. To be asked first, set `"tune_confirm": true` in `~/.ft991a-panel.json`. (Before version 1.1.1 it always asked, and the setting was called `confirm_tune`, which is now ignored.)

Meter corrections

The calibration buttons are hidden until you press the **Calibration** button under the meters (its arrow points down while they are shown). The choice is remembered. The panel shows the values Hamlib reports from its own calibration tables. If a wattmeter disagrees, transmit into a dummy load and press **Cal PO**, then type the true power. Do the same with **Cal SWR**. Each press adds a correction point; the panel interpolates between points, and shows the corrected value with Hamlib's own in brackets. With Raw CAT, press **Cal S9** while receiving a signal you know to be S9 (read it on the radio's own meter); until then the panel assumes S9 is about half scale. **Clear cal** removes the power and SWR points. Points are saved.

What the panel does not do

- It keys the radio only when PTT is on, or while the tuner tunes. FreeDV also keys the radio, so use one keying source at a time.
- It does not set RF or AF gain, because audio travels over the network and RF gain is left at full.
- It has no spectrum display. To my knowledge the FT-991A does not provide scope data over CAT.

Trying it without a radio

Run `python3 fake-rigctld.py --demo` and point the panel at `127.0.0.1`, port 4532. The simulator keys itself for 5 seconds out of every 15, with made-up meter values. For the Raw CAT connection, run python3 fake-ft991a.py --demo instead and use port 4533.

The CW window

The **CW window** is a separate window, opened by the **CW window** button beside the log on every tab (the button reads **Hide CW** while it is open). It sends typed text and macros as Morse, and keys the radio from a **HaliKey MIDI** key or paddles. It remembers where you left it. It works only while the radio is in a CW mode (CW-U or CW-L) and is not transmitting; otherwise it refuses, and says why.

What runs where. The panel does not send the dits and dahs one at a time over the network, because network delays would spoil their timing. A small program on the radio's computer, next to the radio, owns the radio's **second USB serial port** and keys the radio with its DTR line. Two programs can do that, and only one may run at a time, because they conflict over the serial port and over UDP port 6789:

- `cwdaemon` sends text only.
- `keying-daemon.py` (in the repository) answers cwdaemon's text commands on UDP port 6789, so the text box works unchanged, and it also takes timestamped key events on UDP port 6790 from a key or paddles. It plays them back with their original spacing behind a 150 ms jitter buffer, so the Morse on the air is your own, a fixed 150 ms later plus the network delay. Use it if you want to key by hand: it does both jobs.

Setting it up on the radio's computer.

1. Stop cwdaemon if it runs: `sudo systemctl stop cwdaemon` or `sudo pkill cwdaemon`.
2. Let your user open the serial port, so that nothing needs `sudo`: `sudo usermod -aG dialout $USER`, then log out and in again. Use the stable name from `ls /dev/serial/by-id` for the port.
3. On the radio, set menu **060 PC KEYING** to DTR, menu **012 KEYER TYPE** to OFF (the radio's own keyer would answer a held key line with a string of dits) and menu **033 CAT RTS** to DISABLE, and use a CW mode.
4. Install pyserial (`pip install pyserial`) and start the daemon: `python3 keying-daemon.py --device /dev/serial/by-id/YOUR-PORT --bind YOUR-TAILSCALE-ADDRESS --verbose`. The `--verbose` option prints every key change and who asked

for it. The file `keying-daemon.service` runs it in the background. It says what is wrong, and stops, if the serial port or a UDP port is already in use. To try it without a radio, use `--null` instead of `--device`.

5. In the CW window, type the radio's computer in the **cwdaemon on** box (the text port is 6789) and check the **daemon port** (6790). Tick **Arm**, then try **Test key** on a dummy load.

Sending text. Type in the text box and press **Send** (or Enter). The **Speed** slider runs from 5 to 60 wpm. **My call** and **Their call** fill in `[MYCALL]` and `[CALL]` in the five macros F1 to F5, which you can edit; a macro that needs a call you have not typed is refused. Prosigns: `*` is AR, `=` is BT, `<` is SK, `(` is KN, `!` is SN, `&` is AS and `>` is BK; `+` and `-` are left out. The panel says when the message has finished, and what you send is also sounded at the panel as a sidetone, with the same timing. **Abort** stops everything at once: the key lets go, the tone stops, and whatever was pending is forgotten. **Zero in** starts the radio's CW auto zero-in, which tunes the radio to the signal it hears. It moves the radio, so it has the same refusals as sending.

The Key row. Arm tells the daemon that the panel is ready, and sends it a heartbeat every 100 ms. If the heartbeats stop while the key is down, the daemon lets go of the key after 0.8 s, and it never holds the key for more than 3 s. **Test key (hold)** acts as a straight key for as long as you hold the mouse button, so you can check the whole chain on a dummy load.

The HaliKey MIDI key or paddles box. The panel supports one keying device, the HaliKey MIDI. It sends notes 20 (dit) and 21 (dah) and puts its own timing in the messages, which the panel uses.

Control	What it does
Device	**HaliKey paddles** (an iambic keyer), **HaliKey straight key**, or **None**.
Port	The HaliKey's MIDI port. **Refresh** looks at the MIDI ports again, and so does opening the list. The HaliKey is chosen for you when it is there.
Mode	**Iambic A** or **Iambic B**. In B, letting go of both paddles during an element adds the opposite element after it; in A the keyer stops.
Dit memory %	How far into the next element a tap on the other paddle is still remembered. The default is 33.

Control	What it does
Volume	How loud the tone is that you hear while you key or send text, from 0 (silent) to 100. It has nothing to do with the radio or the Audio tab.
Pitch Hz	The pitch of that tone.
SPOT	Holds the tone on, so that you can tune a signal until it matches. It also asks the radio for its CW pitch and uses it, so that matching the tone matches the radio's filter. Press it again to stop.
More	Shows or hides the Monitor and the notes at the bottom of the window. It is remembered, and starts hidden.
Monitor	Shows the notes the HaliKey sends when you press it, in the log, and keys nothing. Use it to see whether the HaliKey is reaching the panel.

Changing a setting while armed (the device, the port, the mode or the dit memory) re-opens the device at once, and the status line names what is armed, for example `HaliKey paddles on HaliKey MIDI: Iambic B`.

The tone. The tone you hear is the panel's own. It plays on the computer's **default sound output**, not on the speakers chosen on the Audio tab, and it starts the moment you press, while the radio keys after the network delay. **The radio's own sidetone** sounds at the radio, and the Function tab's CW page has a **Monitor level (MONI)** slider for it: 0 silences it. On Windows the panel's tone has more delay than on a Mac (see Using the panel on Windows).

MIDI runs in a helper process. python-rtmidi (`pip install python-rtmidi`) is a native library that can crash the program that loads it, and it did on a Mac with Python 3.14. So the panel never loads it. It starts itself a second time as a helper that does all the MIDI work, and talks to it over a socket on the same computer. If the helper dies, the key is let go, the window says why, and the panel carries on: disarm and arm again. If MIDI does not work on your Python, use Python 3.12 or 3.13, or the installer (built with 3.12). Without python-rtmidi the HaliKey is off and everything else works.

The meters while you key. The power, SWR and ALC meters are polled faster while you key, shown as live, and each holds its highest reading for 0.7 s, because the radio is asked only a few times a second and an element can be 50 ms long. They cannot follow single

elements.

Safety. Try the CW window on a dummy load first. Keep the radio's own transmit time-out (menu 036 TX TOT) on. The daemon lets go of the key when the heartbeat stops, when the panel closes or loses the radio, and on Abort. The panel keys only while the radio is in a CW mode.

Tested. The HaliKey MIDI has been used on a real FT-991A over Tailscale, as paddles (iambic B at 20 and 32 wpm) and as a straight key. Text was tried against the real cwdaemon. Serial keys and the Vail adapter are not supported.

Switching the radio on and off

The FT-991A can be switched on and off over its CAT port, so the whole station can be started remotely. This works over the radio's USB connection: its serial port is still present while the radio is off.

The rule

The commands are `PS1;` (on) and `PS0;` (off). The CAT manual says the set command needs dummy data sent first, and the command itself sent between one and two seconds later. In practice: send a stray character such as `;`, wait about 1.4 seconds, then send the command, with nothing else on the line in between. The rule applies to both directions, so a bare `PS0;` is ignored. `PS;` reads the state and answers `PS1;` when the radio is on. In standby the radio answers nothing.

From the server: radio-power.sh

```
bash radio-power.sh on
bash radio-power.sh off
bash radio-power.sh status
```

Stop rigctld and ser2net first, because only one program may use the serial port; the script warns if another program has it open. It checks that the radio answers `ID;`, sends the dummy data, waits, sends the command, and then asks for the power state. Settings come from the environment: `DEV` (default `/dev/yaesu-cat`), `SPEED` (38400, which must match menu 031) and `WAIT` (1.4 seconds; anything from 1 to 2 should work).

From the panel

With the **Raw CAT** connection, **Power on** and **Power off** buttons sit beside Tune. Each press sends the dummy data and, 1.4 seconds later, the command. There is no confirmation, so a press takes effect at once. All other traffic is held back during the wait. Power off is refused while the radio is transmitting. While the radio is off the status reads "No reply (radio off?)", and the panel recovers by itself when the radio comes back. With the Hamlib connection the buttons are disabled, because the panel does not use Hamlib to switch power.

What has been verified

On this installation the radio was switched on and off over USB with dummy data and a wait of about 1.3 seconds. A bare `PS0;` did not switch it off. The panel's Power buttons have been tested on the radio (from macOS, over ser2net). The script and the panel were also tested against a simulated radio that enforces the same timing rule.

A remote start, step by step

1. Switch the radio on, with the script or the panel.
2. Wait a few seconds for its USB audio device and serial port to appear.
3. Start rigctld or ser2net. How rigctld behaves across a power cycle has not been tested.
4. Start the bridge. If the audio device is not there yet, it waits and starts by itself.

When the radio is switched off, its audio device disappears. A bridge that is already running stalls, so press Stop and then Start when the radio is back.

Safety

- Keep the radio's transmit time-out enabled: menu 036 TX TOT, from off up to 30 minutes.
- Do not switch the radio off while it is transmitting.
- Remote operation of a transmitter may carry conditions under your licence.

Using FreeDV and WSJT-X

Both programs use the client's virtual cables for audio, and rigctld for radio control. Run one of them at a time, because they would share the same cables.

FreeDV

- **Audio:** Input From Radio = cable A, Output To Radio = cable B, as in the wiring table. Use PCM for the first tests.

- **Rig control:** choose the rig type **Hamlib NET rigctl** and enter `<server Tailscale address>:4532`. FreeDV's log shows this as `rigName -Hamlib NET rigctl- via port ...`.
- **Mode:** the radio should be in DATA-USB. The panel can set it.
- **Transmit level:** adjust until ALC is idle and power output matches your setting (see Recommended settings).
- **Sample rate:** FreeDV's per-device sample rate should be 48000 Hz, matching the bridge and the BlackHole devices.

FreeDV's own Hamlib client had trouble connecting to the Hamlib 5 snapshot installed here; see Known issues.

WSJT-X

This has not been tested on this installation; these are the settings the design calls for.

- **Audio:** use the same two cables, and PCM at 48000 Hz. Set Soundcard Output to **Mono** or **Both**, because the bridge captures only the left channel.
- **Rig control:** use Hamlib NET rigctl with `<server Tailscale address>:4532`, so WSJT-X, FreeDV and the logger can share one radio connection.
- **Timing:** the bridge adds delay (the jitter buffer plus 10 to 30 ms), so decodes show a small DT offset. FT8 tolerates it, and a smaller jitter buffer reduces it. Keep the operating computer's clock synchronised.
- **Transmit level:** use WSJT-X's power slider and the bridge's gain slider for full power with idle ALC.

Sharing the radio

rigctld lets several programs connect, but the programs can still disagree. A program that sets the mode will override FreeDV's DATA-USB. A logging program should only read the frequency and mode. If a program keeps changing the mode, stop its rig control.

Troubleshooting

Start with the log and the counters in the bridge window. The table lists problems met while setting up this installation, with their causes.

Symptom	Likely cause	What to do
Start does nothing and the terminal prints `invalid argument` (Apple Silicon Mac)	An old engine file. `opuslib` did not tell Python that an Opus function takes variable arguments, which Apple Silicon needs.	Use the current `audio-bridge.py`, which declares it.
"Opus unavailable", PCM only	The Opus library cannot be found.	Windows: put a 64-bit `opus.dll` beside the scripts. macOS: `brew install opus`. Linux: install `libopus0`.
Radio is keyed, but no power and no audio reaches it	Mono playback reached only the left channel of a stereo output.	Keep Playback channels at 2 (see Audio devices). Judge by power output: a flat ALC is normal and says nothing.
Client Capture meter stays flat while FreeDV transmits	FreeDV's Output To Radio is a different cable, FreeDV is not transmitting, microphone permission is missing (macOS), or the BlackHole volume is down.	Check each in turn. The test tone script in Diagnostic tools can feed the cable directly.
Crackling, rising Underruns	Packets arrive unevenly, usually on Wi-Fi.	Raise the jitter buffer to 110 to 200 ms.
High Lost (8 to 14%)	Packets sent unevenly were dropped on the way out of the Mac.	Update both files so that the strict send clock exists, and keep Smooth packet timing on. Check that Send timing shows near-zero too-close. Switch off AirDrop and Handoff on the Mac.
Lost is small but audio still gaps	Single-packet losses with Opus.	Use PCM with a larger jitter buffer, or try In-band FEC with Opus (it helps most at lower bitrates).

Symptom	Likely cause	What to do
Noise reaches FreeDV but nothing decodes (Opus)	Opus adds coding noise, or there is no signal on the frequency.	Compare with PCM on the same signal. For Opus use 64 kbps or more, narrowband, music.
Settings lost after starting with the radio off	Fixed: a fallback device used to overwrite your choice.	Use the current files. Missing devices now show `(not present)` and the bridge waits for them.
Window keeps resizing	Fixed in the current GUI.	Update `audio-bridge-gui.py`.
Audio stops after one end restarts	Fixed: the other end used to discard the restarted sequence numbers.	Update both files. The log shows `Sender restarted - resynchronising audio`.
`Bad` count rising	Sample rate or frame size differ between the ends, or one end cannot decode Opus.	Make rate and frame match; check the log for an Opus warning.
Cannot connect to ser2net or rigctld from the server itself	It is bound to one address only.	Check with `ss -ltnp | grep <port>`. Connect to the address shown, not `localhost`.
rigctld runs but nothing listens on port 4532	It is stuck starting, often because the radio is off or the serial device is wrong.	Run it in the foreground with `-vvv`. Check `/dev/yaesu-cat` and that no other program (such as ser2net) holds the port.
`rigctl` or FreeDV connects, then hangs up	Hamlib client handshake problem in the 5.0 snapshot.	See Known issues. The panel is not affected.

Symptom	Likely cause	What to do
Panel log: "Connection problem (no reply to 't'); reconnecting"	The port does not answer rigctld's commands. It is either a raw ser2net port, or rigctld is waiting for a radio that is switched off or not answering.	The panel's log now says which. For a ser2net port, set Connection to Raw CAT. Otherwise switch the radio on and check rigctld with rigtest.sh.
FreeDV's audio devices change by themselves (for example microphone input and output to radio)	FreeDV started or exited while the radio's USB audio device was missing, for example just after a remote power-on, and saved fallback devices.	Run freedv-guard.sh save while the settings are right. Start FreeDV with freedv-guard.sh run, which waits for the radio's audio and restores the saved devices before and after FreeDV runs. freedv-guard.sh check shows exactly what changed.

Diagnostic tools

These scripts find where a problem is. None of them transmits. Run the network tests with the bridge **stopped** on the machine that is receiving.

udpseq.py: packet loss on the network path

It sends numbered packets and counts the gaps. Plain test, bridge-sized packets at 100 a second for 60 seconds:

```
python3 udpseq.py recv 5900
python3 udpseq.py send <server address> 5900 --size 1000 --pps 100 --seconds
60
```

Press Ctrl-C on the receiver for the summary line. A clean path loses under 0.2%. This installation measured 0.03% to 0.17%.

The staged test finds which bridge behaviour causes loss. It runs six stages of 20 seconds each, and the receiver prints a loss figure per stage:

```
python3 udpseq.py recv 5800 --quiet
python3 udpseq.py send <server address> 5800 --stages --stage-seconds 20 --
device "BlackHole 16ch"
```

Stage	Sending style	Result here
0	Plain, evenly paced	0.10%
1	Socket with a timeout and a reader thread, as the bridge has	0.10%
2	Bursts of two packets	20.4%
3	Triggered by the live audio callback, unpaced	7.1%
4	As 3, with packets at least 8 ms apart	3.3%
5	As 3, on a strict 10 ms clock (what the bridge now does)	0.00%

With the real bridge running on the sending side, `python3 udpseq.py recv 5800 --bridge` counts gaps in its actual traffic (the server bridge must be stopped).

wavstat.py: what was actually played

Tick the bridge's Debug option (or use `--dump-play`), run, stop, then:

```
python3 wavstat.py ~/bridge_play.wav
```

It prints the peak, the share of near-silent blocks, and the level each second. For a steady test tone it also estimates the frequency. Use it to check that audio reaching the radio is continuous and at the right level.

Test tone into a cable

This plays a 1 kHz tone for 15 seconds into a device, to prove that a cable and the bridge's Capture work without FreeDV. Save it as a `.py` file and run it:

```
import math, struct, sounddevice as sd
rate = 48000
dev = "BlackHole 16ch"   # the cable to test
tone = b"".join(struct.pack("<hh", v, v) for v in
                (int(6000 * math.sin(2 * math.pi * 1000 * i / rate)) for i in
range(rate * 15)))
with sd.RawOutputStream(samplerate=rate, channels=2, dtype="int16",
device=dev) as s:
    s.write(tone)
```

rigtest.sh: probe rigctld

`bash rigtest.sh` starts a test rigctld on the server only, reads frequency, mode, PTT, power setting, meters and tuner over plain TCP, lists the radio's capabilities, tries Hamlib's own client, and stops. It **stops any running rigctld first**, so restart yours afterwards. Settings come from the environment: `MODEL` (1035), `DEV` (`/dev/yaesu-cat`), `SPEED` (38400), `PORT` (4532), `HOST` (127.0.0.1), `RIGCTLD` and `RIGCTL` (paths to the programs). `SKIP_START=1 bash rigtest.sh` probes a daemon that is already running, without restarting it. To test a freshly built Hamlib from its source folder, run it with `LD_LIBRARY_PATH=$PWD/src/.libs RIGCTLD=$PWD/tests/rigctld RIGCTL=$PWD/tests/rigctl`.

nettest.sh: Hamlib handshake

It starts rigctld with Hamlib's built-in dummy radio on port 4540, times each step of the handshake, and runs Hamlib's own client against it. If a daemon is already listening on `HOST:PORT`, it does the same against that. A healthy result is a handshake reply in a few milliseconds (about 65 lines for `dump_state`), and the client printing `145000000` and exiting with code 0.

fake-rigctld.py: a stand-in radio

A simulated rigctld for trying the panel without a radio: `python3 fake-rigctld.py --demo`.

Paste a script's output into a chat, never back into a terminal.

Software license

FT-991A Companion is free software under the **GNU General Public License, version 3 or later**. You may use it, study it, change it and share it, on the same terms, and it comes with no warranty. The full text is in the file `LICENSE`, the credits are in `NOTICE.md`, and the licences of the software that the installers include (Python, Tcl/Tk, numpy, sounddevice, PortAudio, opuslib and the Opus codec, built with PyInstaller) are in `THIRD-PARTY-LICENSES.txt`.

Every installer and packaged program has these three files inside it. Run the program with `--license` for the notice, with `--license full` to print the GPL, or with `--save-license FOLDER` to copy the three files into a folder. The Windows program has no

console, so use `--save-license` there. The Windows installer shows the GPL on its licence page and installs the files, and the macOS disk image contains them beside the app.

The source code of every version is the project's repository, https://github.com/cwpowell/FT-991a-Companion. Each release is tagged with its version number, and the tag holds the source that was built into that release's installers.

Versions before 1.1.0 were released under the MIT License, and a copy of one of those that you already have stays under it.

Known issues and limits

The audio link has been measured on this installation and works. The panel has been tested on a real FT-991A from macOS, and Linux, and on Windows for basic operation. The items below are open or untested as of this version.

Item	Status	Workaround
Hamlib client handshake	In the installed Hamlib 5.0 development build (`11ff5246f`, 2026-09-22), `rigctl -m 2` and FreeDV's own client connect, receive the handshake replies, and then hang up; rigctld logs `Connection reset by peer`. A newer build (`fa0ed6d6a`, compiled 2026-10-19) completes the handshake against Hamlib's dummy radio; it has not yet been tested against the FT-991.	The panel and plain TCP are unaffected, and the panel's Raw CAT connection (through ser2net) avoids rigctld altogether. Run `nettest.sh` against the new build. If clients still fail, use a stable Hamlib 4.x: the FT-991 backend (model 1035) is in 4.5.5, and on Debian or Ubuntu `sudo apt install libhamlib-utils` installs it as `/usr/bin/rigctld` without touching `/usr/local`.
FreeDV and WSJT-X through rigctld	FreeDV could not connect while the handshake problem existed. WSJT-X has not been tried.	See the row above.

Item	Status	Workaround
Opus with FreeDV	With the original Opus settings (32 kbps, full bandwidth) FreeDV showed noise but did not decode. The new defaults are better on a test signal but have not been compared with PCM on a live FreeDV signal.	Use PCM.
Radio switched off while the bridge runs	The audio streams stall and the bridge does not recover by itself.	Press Stop, then Start when the radio is back.
Panel on the real radio	Not yet used with the real radio. Unverified: whether Stop tune (PTT off) halts the radio's tuner, whether `TUNE` blocks rigctld while it runs, and whether Hamlib's power and SWR scales match a wattmeter.	Keep a hand near the radio's TUNE key the first time; use Cal PO and Cal SWR to correct the meters.
rigctld at boot	No service definition has been written.	Start it by hand, detached from the terminal.
Windows	The bridge's Windows handling of `opus.dll` was not run on Windows. The panel's own Windows test used PCM, so Opus on Windows is untested.	Check the log for the Opus line on start.
Security	No authentication or encryption on the bridge or rigctld.	Private network only (see Network and security).
Spectrum display	The panel shows no scope or waterfall. To my knowledge the FT-991A does not provide it over CAT.	Use the radio's own display.

Item	Status	Workaround
Clock drift	The two sound clocks differ slightly, causing an occasional underrun.	Harmless; FreeDV tolerates it.
Remote power on and off	Verified on the radio by hand (dummy data, then the command after about 1.3 s). The panel's Power buttons have been tested on the radio (from macOS, over ser2net), and the script against a simulated radio. How rigctld behaves across a power cycle is untested.	Stop rigctld and ser2net before using the script. Watch the first power-off with a hand near the POWER key.
Split and swapping the VFOs	The CAT manual has no active-VFO or split command, only FT (which VFO transmits), so the Split button sets the transmit VFO and clicking VFO B swaps the two VFOs with SV. The Hamlib connection's split uses rigctld's S and s commands. Split and the VFO swap are tested on the radio with the Raw CAT connection (from macOS, over ser2net). The Hamlib connection's split was tested only against a simulated rigctld.	Try it into a dummy load first, and check for split operation. The radio's own A/B key remains the reference.
S meter scale (Raw CAT)	The raw 0 to 255 reading is converted to S units with an assumed S9 at 128. The true point is not in the CAT manual.	Press Cal S9 while receiving a known S9 signal.

Item	Status	Workaround
Frequency entry, digit tuner, C4FM and bandwidth	Tested on the radio with the Raw CAT connection. In data modes the slider assumes menu 062 OTHER uses the SSB width table; the manual does not say.	Compare the slider's width with the radio's own display, and try a frequency change on a dummy load first.
Audio tab	Tested on a real FT-991A with real sound devices, on macOS, in both directions. On Windows, tested for basic operation: the radio and the audio connect by themselves, and the CW sidetone has more delay. Receive volume and Mic level are software gains on this computer.	Check the first transmission into a dummy load and watch the ALC and power. Keep Mic OFF until the radio is keyed.
Audio and the bridge window	The server sends audio to one client at a time. Starting audio in the panel takes the stream away from the bridge window used for FreeDV, and the reverse.	Close one before starting the other.
PTT and other keying sources	The panel can key the radio with its PTT button. FreeDV and other programs can key it too, so two keying sources can conflict. Tested on the radio (from macOS, over ser2net).	Use one keying source at a time. PTT lets go when you turn it off, after 120 seconds, and on disconnect or close, but not when you click another window. Keep the radio's own TX time-out (menu 036) enabled too.

Item	Status	Workaround
Two copies of the audio engine	The panel contains its own copy of the bridge's audio engine, and the server runs audio-bridge.py. They speak the same protocol, which has not changed, so either one can be updated on its own. An older server was tested with the new panel, for PCM and Opus. The server file from 2026-10-25a adds recovery when the radio is switched off and on, which the panel does not need.	Keep the panel and the server's bridge from the same download.
Routing between sound devices	The receive and transmit paths join two sound devices that each have their own clock, so over a long time the panel drops or repeats a few milliseconds, shown as dropped audio and underruns. In remote mode the received audio and your microphone both pass through the panel in memory, so the virtual cables add no delay to them. Only audio that FreeDV sends through the cable has the cable's buffering.	Use "(direct)" when FreeDV is not using the cables.

Item	Status	Workaround
Radio ends in remote mode	Radio to computer must be "(direct)" or a virtual cable the panel can play into, and Computer to radio must be "(direct)" or a virtual cable it can record from. BlackHole is both. Windows cables such as VB-Cable have a separate playback side and record side. A real sound card is refused there.	Use Local mode for a sound card that is on this computer.
macOS prints "PaMacCore (AUHAL)" errors in Terminal	Core Audio can refuse to open one device for playing and recording in the same program (errors -10863 and -50). On the receive side the panel avoids this by not reading the cable back. The transmit side avoids it too: your microphone goes to the bridge in memory, and the panel opens the radio cable only once, to read FreeDV's audio. The panel now catches these messages and lists them in its log with the device that caused them, instead of printing them in Terminal.	If the audio works, the messages can be ignored.

Quick reference

This installation

Item	Value
Server	Linux, with the FT-991A on USB. Tailscale address `100.x.y.z`.
Client	macOS on Apple Silicon, Python 3.14.3. BlackHole 2ch is cable A, BlackHole 16ch is cable B.
Audio bridge	UDP port 5800. PCM, 48000 Hz, 10 ms frames, jitter buffer 110 ms. Server TX gain -9 dB.
rigctld	TCP port 4532, model 1035, serial speed 38400 on `/dev/yaesu-cat`.
Bridge version	2026-10-26a

Commands

Task	Command
Bridge window	`python3 audio-bridge-gui.py`
List audio devices	`python3 audio-bridge.py list`
Bridge, command line, waiting for the radio	`python3 audio-bridge.py server --wait --in-dev "<name>" --out-dev "<name>"`
Start rigctld	`rigctld --model=1035 --rig-file=/dev/yaesu-cat --port=4532 --serial-speed=38400 -T <address>`
Panel	`python3 ft991a-panel.py --host <address> --port 4532`
Frequency over plain TCP	`printf 'f\n' | nc -w 2 <address> 4532`
Loss test	`python3 udpseq.py recv 5900` and `python3 udpseq.py send <address> 5900`
Probe rigctld	`bash rigtest.sh` (stops a running rigctld first)
Radio power on or off	`bash radio-power.sh on` or `off` (stop rigctld and ser2net first)

Task	Command
python3 keying-daemon.py --device /dev/serial/by-id/<port> --bind <address> --verbose	On the radio's computer: the keying daemon for the CW window (add --null instead of --device to try it without a radio).

Healthy numbers

Counter	Target
Lost	Under 0.5% of RX packets
Late, TX errors, Bad, Dropped	0
Send timing, too-close	Near 0
Underruns	A few per hour at most

Files written

File	Contents
`~/.audio-bridge.json`	Bridge window settings and intended devices
`~/.ft991a-panel.json`	Panel host, port, window position, meter corrections
`~/bridge_play.wav`, `~/bridge_capture.wav`	Debug recordings, when the Debug option is on
`/tmp/rigctld_test.log`, `/tmp/rig_caps.txt`	Written by `rigtest.sh`